Fundamentos del aprendizaje de máquina

M6: Aprendizaje de máquina supervisado

|**AE1**: Explicar conceptos fundamentales del aprendizaje de máquina supervisado para la resolución de un problema distinguiendo entre tareas de clasificación y de regresión.

alkemy

Introducción 🧑‍💻

El aprendizaje de máquina (Machine Learning) se ha convertido en uno de los pilares fundamentales de la transformación digital y la inteligencia artificial moderna. Este enfoque permite que los sistemas informáticos aprendan de los datos, mejoren su rendimiento y tomen decisiones basadas en patrones sin intervención humana explícita. Cada día, millones de personas interactúan con tecnologías impulsadas por aprendizaje de máquina, desde recomendaciones de productos hasta asistentes virtuales y sistemas de detección de fraudes.

El propósito de este manual es proporcionar una base sólida para comprender los principios esenciales del aprendizaje de máquina. Aquí exploraremos sus aplicaciones, los tipos de aprendizaje existentes, y las etapas comunes que conforman el proceso de modelado. También profundizaremos en los algoritmos más utilizados para tareas de regresión y clasificación, dos de las problemáticas más comunes en este campo.

Este documento está diseñado para estudiantes, profesionales y entusiastas que desean adquirir un conocimiento estructurado y práctico de los fundamentos del aprendizaje de máquina, sirviendo como base para proyectos, certificaciones y desafíos técnicos del mundo real.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Comprender los conceptos básicos y la definición del aprendizaje de máquina, así como su importancia en el contexto actual de la tecnología y los datos.
- Identificar las principales aplicaciones del aprendizaje de máquina en distintos sectores e industrias.
- Reconocer y explicar las etapas típicas que conforman un proceso de aprendizaje de máquina, desde la recolección de datos hasta la evaluación del modelo.
- Diferenciar los tipos de aprendizaje de máquina: supervisado, no supervisado y sus variantes, aplicando ejemplos prácticos.
- Analizar y comprender los algoritmos más utilizados en tareas de regresión y clasificación, evaluando sus ventajas y limitaciones.

Fundamentos del Aprendizaje de Máquina

1. ¿Qué es el Aprendizaje de Máquina?

El aprendizaje de máquina (Machine Learning) es una rama de la inteligencia artificial que permite a los sistemas informáticos aprender y mejorar automáticamente a partir de la experiencia, sin ser explícitamente programados para cada tarea. Este enfoque consiste en el desarrollo de algoritmos capaces de identificar patrones dentro de grandes volúmenes de datos y, a partir de ellos, realizar predicciones o tomar decisiones.

A diferencia de los sistemas tradicionales basados en reglas predefinidas, los modelos de aprendizaje de máquina adaptan su comportamiento en función de los datos que reciben. Esto significa que cuanto mayor sea la cantidad y calidad de los datos disponibles, mejor será el desempeño del modelo, ya que podrá generalizar y predecir con mayor precisión nuevos casos.

Uno de los aspectos clave del aprendizaje de máquina es que no requiere que un programador conozca todas las reglas para resolver un problema; el sistema las infiere directamente a partir de los datos. Por ejemplo, para reconocer imágenes de gatos, un algoritmo de aprendizaje de máquina no necesita que alguien le indique las características de un gato; en cambio, analiza miles de imágenes etiquetadas y aprende a identificar patrones comunes.

El proceso de aprendizaje se basa en la construcción de un modelo matemático, el cual ajusta sus parámetros internos mediante un proceso iterativo. Estos modelos pueden ser simples, como una regresión lineal, o complejos, como las redes neuronales profundas. El aprendizaje de máquina se ha convertido en una herramienta fundamental en la era de los datos masivos, impulsando desde asistentes virtuales hasta sistemas de recomendación.

Existen diferentes enfoques y técnicas de aprendizaje de máquina, que varían según la naturaleza de los datos y el problema que se desea resolver. Algunos métodos requieren datos etiquetados, mientras que otros trabajan con datos sin etiquetar, y hay técnicas mixtas que combinan ambos tipos.

Es importante destacar que el aprendizaje de máquina no garantiza resultados perfectos. Los modelos pueden cometer errores, especialmente si los datos de

entrada están incompletos, desbalanceados o contienen ruido. Por ello, es crucial un proceso cuidadoso de preparación y evaluación de datos.

En resumen, el aprendizaje de máquina es una disciplina esencial para transformar datos en valor. Su aplicación responsable y ética contribuye a resolver problemas complejos, optimizar procesos y generar conocimiento a partir de la información disponible.

2. Aplicaciones del Aprendizaje de Máquina

El aprendizaje de máquina ha transformado diversos sectores gracias a su capacidad para analizar grandes volúmenes de datos y generar conocimiento accionable. Sus aplicaciones son cada vez más frecuentes en la vida cotidiana, en el ámbito empresarial, científico y social, impactando desde la automatización de tareas hasta la toma de decisiones complejas basadas en datos.

Uno de los ámbitos más populares donde se utiliza el aprendizaje de máquina es en los sistemas de recomendación. Plataformas como Netflix, Amazon y Spotify aplican algoritmos de aprendizaje supervisado y no supervisado para sugerir productos, películas o música según los gustos y comportamientos anteriores de los usuarios.

Otra aplicación clave es la detección de fraudes en transacciones financieras. Mediante el análisis de patrones históricos de comportamiento, los modelos pueden identificar actividades inusuales o sospechosas, enviando alertas en tiempo real para prevenir pérdidas económicas.

En el campo de la salud, el aprendizaje de máquina se ha consolidado como una herramienta poderosa para el diagnóstico asistido por computadora. Algoritmos de clasificación son utilizados para detectar enfermedades a partir de imágenes médicas, como radiografías, resonancias magnéticas o análisis de laboratorio, mejorando la precisión y rapidez en los diagnósticos.

También es fundamental en los sistemas de reconocimiento facial y biometría, los cuales son ampliamente utilizados en aplicaciones de seguridad, control de acceso y desbloqueo de dispositivos móviles. Estos sistemas aprenden a identificar rostros, huellas dactilares o patrones de voz a partir de grandes bases de datos.

En el sector industrial y de manufactura, el aprendizaje de máquina se aplica en el mantenimiento predictivo de maquinaria. Los algoritmos analizan señales y

registros históricos para anticipar posibles fallos, evitando interrupciones costosas en la producción.

Asimismo, en el ámbito del transporte, los modelos de aprendizaje de máquina son utilizados para optimizar rutas, predecir el tráfico o incluso para el desarrollo de vehículos autónomos. Estos vehículos integran múltiples algoritmos que analizan constantemente el entorno y toman decisiones en tiempo real.

Finalmente, en el campo de los negocios y el marketing, el aprendizaje de máquina permite segmentar clientes, predecir tendencias de compra y automatizar procesos de atención al cliente mediante chatbots, incrementando la eficiencia y personalización de los servicios.

Como puede observarse, el aprendizaje de máquina tiene un campo de aplicación muy amplio y en constante crecimiento. Su implementación adecuada ofrece ventajas competitivas significativas a las organizaciones y una mejor experiencia para los usuarios finales.

3. Etapas Típicas de un Problema de Aprendizaje de Máquina

El desarrollo de un proyecto de aprendizaje de máquina sigue una serie de etapas fundamentales que permiten estructurar y garantizar el éxito del proceso. Estas etapas son clave para transformar los datos en modelos predictivos o clasificatorios útiles y confiables.

- La primera etapa es la **definición del problema**. En esta fase se debe entender claramente cuál es el objetivo del proyecto y qué pregunta o necesidad se quiere responder mediante el aprendizaje de máquina. Es esencial establecer si se trata de un problema de clasificación, regresión, agrupamiento o recomendación.

- Posteriormente, se lleva a cabo la **recolección y preparación de los datos**. Aquí se identifican las fuentes de datos relevantes y se procede a la recopilación de información. Una vez obtenidos los datos, se realiza la limpieza, eliminación de duplicados, tratamiento de valores nulos y normalización, garantizando que los datos sean aptos para el entrenamiento de los modelos.

- La siguiente fase es el **análisis exploratorio de los datos (EDA - Exploratory Data Analysis)**. En este paso, se analiza la distribución de los

datos, se identifican relaciones, patrones y posibles outliers. Esta etapa es crucial para comprender las características de los datos y guiar la selección de las técnicas y modelos a utilizar.

- Una vez entendido el conjunto de datos, se pasa a la **selección del modelo y técnica de aprendizaje**. Dependiendo del problema, se elige un algoritmo supervisado o no supervisado. También se decide el tipo de tarea (clasificación, regresión, clustering) y la métrica de evaluación más adecuada para medir el desempeño del modelo.

- Luego, se realiza la **división del conjunto de datos**. Los datos se separan en conjuntos de entrenamiento, validación y prueba. Esta división permite evaluar el comportamiento del modelo y evitar problemas de sobreajuste, asegurando que el modelo generalice correctamente a nuevos datos.

- En la siguiente etapa, se lleva a cabo el **entrenamiento y ajuste del modelo**. Aquí, el modelo aprende a partir de los datos de entrenamiento. Además, se realizan ajustes de hiperparámetros y se prueba con los datos de validación para mejorar la precisión y evitar errores.

- Finalizado el entrenamiento, se realiza la **evaluación del modelo**. Se utilizan métricas específicas como precisión, recall, F1-score, RMSE o MAE, según corresponda, para medir el rendimiento del modelo utilizando el conjunto de prueba.

- Finalmente, una vez validado el modelo, se procede a la **implementación y monitoreo en producción**. El modelo se integra en un entorno real para su uso continuo, y es fundamental establecer un monitoreo constante para identificar posibles degradaciones de rendimiento y actualizarlo según sea necesario.

Estas etapas forman un ciclo continuo de mejora y ajuste, ya que los modelos deben adaptarse a los cambios en los datos y necesidades del negocio.

4. Tipos de Aprendizaje de Máquina

El aprendizaje de máquina se puede clasificar en diferentes tipos, dependiendo de la forma en que el algoritmo recibe la información y aprende de los datos. Esta clasificación permite seleccionar el enfoque más adecuado para cada problema, considerando la naturaleza de los datos y el objetivo del análisis.

El primer tipo es el **aprendizaje supervisado**. En este enfoque, los datos de entrada están etiquetados, es decir, cada conjunto de datos incluye tanto las características como el resultado esperado. El modelo aprende a partir de estos datos etiquetados para predecir resultados en datos nuevos. Ejemplos típicos incluyen la predicción de precios de viviendas, la detección de fraudes y la clasificación de correos electrónicos como spam o no spam.

En segundo lugar, encontramos el **aprendizaje no supervisado**. En este caso, los datos no están etiquetados, por lo que el modelo debe encontrar patrones y estructuras ocultas dentro de los datos sin una guía explícita. Los problemas más comunes son la agrupación o clustering y la reducción de dimensionalidad. Un ejemplo de aplicación es la segmentación de clientes basada en comportamientos de compra.

Existe también un enfoque intermedio conocido como **aprendizaje semi-supervisado**, en el cual el modelo se entrena con un conjunto reducido de datos etiquetados y una gran cantidad de datos no etiquetados. Este enfoque es útil cuando el etiquetado manual de los datos es costoso o complejo, pero existe gran cantidad de datos sin clasificar.

Otro tipo relevante es el **aprendizaje por refuerzo**. En este método, el modelo aprende mediante la interacción con un entorno. No recibe etiquetas, sino recompensas o penalizaciones según las acciones que realiza. Este tipo de aprendizaje es utilizado en áreas como la robótica, videojuegos y sistemas de recomendación en tiempo real.

Dentro del aprendizaje supervisado, se pueden identificar dos grandes familias de tareas: **regresión** y **clasificación**. La regresión busca predecir un valor numérico continuo, mientras que la clasificación intenta asignar un dato a una categoría o clase específica.

En el caso del aprendizaje no supervisado, el enfoque más común es el **clustering**, que consiste en agrupar elementos similares, y la **reducción de dimensionalidad**, que permite simplificar los datos eliminando información redundante.

El aprendizaje por refuerzo, por su parte, se basa en un proceso continuo de prueba y error. El modelo toma decisiones, recibe un feedback positivo o negativo y ajusta su comportamiento en consecuencia, optimizando sus acciones para maximizar la recompensa a largo plazo.

Comprender los distintos tipos de aprendizaje de máquina es fundamental para elegir la estrategia adecuada en función del problema y los datos disponibles, garantizando así resultados más efectivos y precisos.

5. Aprendizaje Supervisado vs No Supervisado

El aprendizaje supervisado y el aprendizaje no supervisado son dos de las categorías fundamentales dentro del aprendizaje de máquina, y comprender sus diferencias es clave para decidir qué enfoque aplicar según el problema que se desea resolver.

- El **aprendizaje supervisado** se basa en el uso de datos etiquetados. Esto significa que, para cada entrada, se conoce de antemano el resultado esperado. El objetivo del modelo es aprender la relación entre las variables de entrada y la etiqueta o resultado para poder realizar predicciones futuras. Un ejemplo clásico es la detección de correos electrónicos como spam, donde se entrena un modelo con ejemplos de correos etiquetados como "spam" o "no spam".

- Por el contrario, el **aprendizaje no supervisado** trabaja con datos sin etiquetar. El algoritmo debe analizar los datos y descubrir patrones, relaciones o estructuras ocultas sin que se le indique explícitamente qué buscar. Un caso típico es la segmentación de clientes basada en sus patrones de compra, donde no existen categorías predefinidas y el algoritmo identifica grupos con características similares.

En términos de complejidad, el aprendizaje supervisado suele requerir mayor esfuerzo en la preparación de los datos, ya que es necesario contar con una cantidad significativa de datos etiquetados. Sin embargo, una vez entrenado, el modelo ofrece resultados más precisos para tareas concretas. En cambio, el aprendizaje no supervisado permite descubrir información nueva y desconocida, aunque los resultados pueden ser menos predecibles o más difíciles de interpretar.

Un aspecto importante es el tipo de tareas que cada enfoque permite resolver. En el aprendizaje supervisado, las tareas más comunes son la **clasificación** y la **regresión**. La clasificación consiste en asignar una categoría a cada entrada, mientras que la regresión busca predecir un valor numérico. Por su parte, en el aprendizaje no supervisado, las tareas más frecuentes son el **clustering** (agrupamiento) y la **reducción de dimensionalidad**.

Otra diferencia clave radica en la evaluación de los resultados. En aprendizaje supervisado, es posible medir la precisión del modelo utilizando métricas objetivas, ya que se conocen las etiquetas verdaderas. En aprendizaje no supervisado, en cambio, la evaluación es más subjetiva, pues no existen resultados "correctos" con los que comparar.

En cuanto a los recursos necesarios, el aprendizaje supervisado requiere un conjunto de datos más elaborado y costoso de obtener, ya que implica la labor de etiquetar cada entrada. El aprendizaje no supervisado es más flexible, al poder trabajar directamente con los datos sin etiquetar, aunque el análisis y validación de los resultados pueden demandar un mayor esfuerzo.

Visualización referencial:

```
import matplotlib.pyplot as plt

import numpy as np


# Datos simulados

X_supervised = np.random.rand(50, 2)

y = ['Clase A' if x[0] + x[1] > 1 else 'Clase B' for x in X_supervised]

X_unsupervised = np.random.rand(50, 2)


# Gráfico supervisado

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

for label in np.unique(y):

    xi = X_supervised[np.array(y) == label]

    plt.scatter(xi[:, 0], xi[:, 1], label=label)

plt.title("Aprendizaje Supervisado")

plt.legend()
```

```
# Gráfico no supervisado

plt.subplot(1, 2, 2)

plt.scatter(X_unsupervised[:, 0], X_unsupervised[:, 1], color='gray')

plt.title("Aprendizaje No Supervisado")


plt.show()
```

6. Algoritmos regresivos

Los **algoritmos regresivos** son un tipo de algoritmos de *aprendizaje supervisado* que se utilizan en *Machine Learning* y estadística para **predecir un valor continuo** en lugar de una clase o categoría. Su objetivo principal es **encontrar relaciones entre una variable dependiente (la que queremos predecir) y una o más variables independientes (las que usamos para predecirla)**.

Características clave de los algoritmos regresivos:

1. **Salida continua**: A diferencia de los algoritmos de clasificación que predicen etiquetas (como "spam" o "no spam"), los algoritmos regresivos predicen valores numéricos, como el precio de una casa, la temperatura, o los ingresos mensuales.

2. **Modelan relaciones**: Intentan modelar la relación matemática entre variables. Por ejemplo, cómo el tamaño de una casa (variable independiente) afecta su precio (variable dependiente).

3. **Aplican funciones matemáticas**: Muchos algoritmos regresivos utilizan funciones lineales, polinomiales o incluso más complejas para ajustar los datos y hacer predicciones.

Característica	Descripción

Salida continua	Predicen números reales: precios, temperaturas, ingresos, etc.
Relación entre variables	Modelan cómo las variables independientes afectan a la dependiente.
Funciones matemáticas	Utilizan relaciones lineales, polinomiales o complejas para ajustar los datos.

Visualización referencial: Regresión lineal simple

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression


# Datos simulados

X = np.array([[40], [50], [60], [70], [80], [90], [100]])

y = np.array([80, 150, 200, 250, 300, 320, 350])


# Modelo

model = LinearRegression()

model.fit(X, y)

y_pred = model.predict(X)


# Gráfico

plt.scatter(X, y, color='blue', label='Datos reales')

plt.plot(X, y_pred, color='red', label='Regresión lineal')

plt.title("Ejemplo de Regresión Lineal")

plt.xlabel("Tamaño (m²)")
```

```
plt.ylabel("Precio (miles)")

plt.legend()

plt.grid(True)

plt.show()
```

7. ¿Qué es una tarea de regresión?

En el contexto del aprendizaje supervisado, una **tarea de regresión** consiste en predecir un valor numérico y continuo a partir de un conjunto de variables o características de entrada. A diferencia de las tareas de clasificación, donde el objetivo es predecir una categoría o clase, en la regresión el resultado es siempre un número real dentro de un rango posible.

📌 **Ejemplos de tareas de regresión:**

- Predicción del precio de un inmueble.
- Estimación de la demanda de un producto.
- Cálculo de la temperatura futura en una región.
- Proyección de ingresos anuales de una empresa.
- Predicción de la duración de un proceso industrial.

💡 Las tareas de regresión son esenciales para problemas donde el objetivo es cuantificar un resultado, y suelen ser la base de análisis financieros, meteorológicos, de salud, logística, entre otros.

Características clave de una tarea de regresión

Característica	Descripción
Tipo de salida	Numérica y continua.
Datos de entrada	Variables independientes (números, categorías, fechas, etc.).
Objetivo	Predecir una cantidad.
Ejemplos típicos	Precio, tiempo, cantidad, distancia, velocidad, entre otros.
Métricas de evaluación	Error Cuadrático Medio (MSE), Error Absoluto Medio (MAE), R^2.

8. Algoritmos más utilizados

A lo largo de los años, se han desarrollado múltiples algoritmos para resolver problemas de regresión. A continuación, se detallan los más relevantes:

🔹 **Regresión Lineal**

Es el algoritmo más simple y utilizado como base en muchos problemas de predicción. Busca establecer una relación lineal entre una variable dependiente y una o varias variables independientes.

Fórmula básica:

$y=mx+b$

donde:

- yyy es la variable objetivo.
- mmm es la pendiente.
- xxx es la variable independiente.
- bbb es el término independiente.

✅ **Ventajas:**

- Fácil de interpretar.
- Computacionalmente eficiente.
- Bueno para relaciones lineales simples.

❌ **Limitaciones:**

- No capta relaciones no lineales.
- Sensible a valores atípicos.

🔹 **Regresión Polinómica**

Es una extensión de la regresión lineal que permite capturar relaciones no lineales mediante la incorporación de términos polinomiales.

Ejemplo de ecuación:

$y=b0+b1x+b2x2+b3x3+\cdots+bnxn$

✅ **Ventajas:**

- Aumenta la capacidad de ajuste frente a datos no lineales.

❌ **Limitaciones:**

- Riesgo de sobreajuste si el grado del polinomio es muy alto.
- Interpretación menos intuitiva.

🔹 **Árboles de Decisión para Regresión**

Dividen el espacio de entrada en segmentos y ajustan un modelo simple (como la media) en cada segmento. Cada nodo del árbol representa una decisión basada en una característica del conjunto de datos.

✅ **Ventajas:**

- Capturan relaciones no lineales.
- Interpretación sencilla.

❌ **Limitaciones:**

- Sensibles a pequeñas variaciones en los datos.
- Pueden sobreajustarse.

🔹 **Random Forest Regressor**

Es un ensamble de árboles de decisión que mejora la precisión y estabilidad de los resultados, reduciendo el sobreajuste.

✅ **Ventajas:**

- Alta precisión.
- Maneja bien datos faltantes y escalas diferentes.

❌ **Limitaciones:**

- Difícil de interpretar.
- Elevado costo computacional.

🔹 **Redes Neuronales para Regresión**

Las redes neuronales pueden configurarse para realizar tareas de regresión ajustando su función de pérdida y salida.

✅ **Ventajas:**

- Capturan relaciones complejas y no lineales.
- Escalables para grandes volúmenes de datos.

❌ **Limitaciones:**

- Requieren grandes cantidades de datos.
- Mayor complejidad en la implementación y ajuste.

Algoritmo	Relación que modela	Tolerancia a datos atípicos	Complejidad	Interpretabilidad
Regresión Lineal	Lineal	Baja	Baja	Alta
Regresión Polinómica	No lineal simple	Baja	Media	Media
Árbol de Decisión	No lineal segmentada	Media	Baja	Alta
Random Forest Regressor	No lineal compleja	Alta	Alta	Baja
Redes Neuronales	No lineal compleja	Media/Alta	Muy alta	Muy baja

9. Algoritmos de Clasificación

¿Qué es una tarea de clasificación?

La **clasificación** es un tipo de problema en el aprendizaje supervisado donde el objetivo es predecir una categoría o clase para un conjunto de datos de entrada. A diferencia de las tareas de regresión, que predicen un valor numérico continuo, en clasificación el resultado es una etiqueta discreta.

📌 **Ejemplos de tareas de clasificación:**

- Determinar si un correo electrónico es spam o no (clasificación binaria).
- Clasificar un conjunto de imágenes según la categoría (perro, gato, pájaro).
- Detectar el sentimiento de un comentario como positivo, neutro o negativo.
- Asignar etiquetas múltiples a un documento (clasificación multi-etiqueta).

Este tipo de tareas es fundamental en áreas como la seguridad informática, la medicina, las finanzas y el marketing digital.

10. Tipos de tarea de clasificación

Existen diferentes tipos de problemas de clasificación según la cantidad y naturaleza de las etiquetas:

🔹 **Clasificación Binaria**

Consiste en predecir entre dos posibles clases.
Ejemplos:

- Aprobado / Reprobado
- Spam / No spam
- Cliente que compra / Cliente que no compra

Gráfico de dispersión con frontera de decisión entre dos clases.

```
from sklearn.datasets import make_classification

from sklearn.linear_model import LogisticRegression
```

```
import matplotlib.pyplot as plt

import numpy as np


# Datos simulados

X, y = make_classification(n_samples=100, n_features=2, n_redundant=0, n_classes=2, random_state=42)

model = LogisticRegression()

model.fit(X, y)


# Frontera de decisión

x_min, x_max = X[:, 0].min()-1, X[:, 0].max()+1

y_min, y_max = X[:, 1].min()-1, X[:, 1].max()+1

xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))

Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)


plt.contourf(xx, yy, Z, alpha=0.3)

plt.scatter(X[:, 0], X[:, 1], c=y, cmap='bwr', edgecolor='k')

plt.title("Clasificación Binaria: Spam / No spam")

plt.xlabel("Característica 1")

plt.ylabel("Característica 2")

plt.show()
```

✅ **Ventajas:** Sencilla de implementar y evaluar.

❌ **Limitaciones:** No aplicable para problemas con múltiples categorías.

🔹 **Clasificación Multiclase**

Aquí el objetivo es asignar una única clase a cada instancia, pero hay más de dos categorías posibles.

Ejemplos:

- Clasificación de tipos de vehículos: coche, camión, moto, bicicleta.
- Detección del estado de un dispositivo: encendido, apagado, en espera.

Gráfico de dispersión 2D con 3 clases codificadas por color:

```
from sklearn.datasets import make_blobs

from sklearn.linear_model import LogisticRegression

import matplotlib.pyplot as plt


X, y = make_blobs(n_samples=150, centers=3, random_state=0, cluster_std=1.2)

model = LogisticRegression(multi_class='ovr')

model.fit(X, y)


plt.scatter(X[:, 0], X[:, 1], c=y, cmap='Set1', edgecolor='k')

plt.title("Clasificación Multiclase: Vehículos")

plt.xlabel("Característica 1")

plt.ylabel("Característica 2")

plt.show()
```

✅ **Ventajas:** Útil para problemas con múltiples clases excluyentes.

❌ **Limitaciones:** La complejidad crece con la cantidad de clases.

🔹 **Clasificación Multi-etiqueta**

Permite asignar varias clases simultáneamente a una instancia.

Ejemplos:

- Etiquetado de imágenes con múltiples objetos: "persona", "bicicleta", "árbol".
- Clasificación de un documento en varias categorías temáticas.

Mapa binario de etiquetas para diferentes instancias (tipo heatmap).

```
import seaborn as sns

import pandas as pd

import matplotlib.pyplot as plt


multi_label_data = pd.DataFrame({

    "Imagen 1": [1, 0, 1, 0],

    "Imagen 2": [0, 1, 1, 0],

    "Imagen 3": [1, 1, 0, 1],

    "Imagen 4": [1, 0, 1, 1]

}, index=["Persona", "Perro", "Bicicleta", "Árbol"])


sns.heatmap(multi_label_data, annot=True, cmap="YlGnBu", cbar=False)

plt.title("Clasificación Multi-etiqueta: Etiquetas por Imagen")

plt.xlabel("Instancias")

plt.ylabel("Etiquetas")

plt.show()
```

✅ **Ventajas:** Aplicable cuando los datos pueden pertenecer a varias categorías.
❌ **Limitaciones:** Requiere técnicas específicas y mayor capacidad computacional.

Resumen de utilidad visual

Tipo de clasificación	Visualización ideal	Objetivo didáctico
Binaria	Dispersión con frontera	Entender separación entre dos clases
Multiclase	Dispersión por colores	Visualizar más de dos categorías
Multi-etiqueta	Heatmap de etiquetas por fila	Mostrar asignaciones múltiples simultáneas

11. Algoritmos más utilizados

A continuación, se detallan los principales algoritmos utilizados en problemas de clasificación:

🔸 **Regresión Logística**

Pese a su nombre, se utiliza para clasificación binaria. Calcula la probabilidad de pertenencia a una clase.

Características:

- Basado en función sigmoide.
- Fácil de interpretar.
- Recomendado para datos linealmente separables.

🔸 **Árboles de Decisión**

Construyen un modelo basado en decisiones binarias sobre las características de entrada.

Características:

- Intuitivos y visuales.
- Soportan datos numéricos y categóricos.
- Pueden sobreajustar si no se podan adecuadamente.

🔸 Random Forest Classifier

Conjunto de árboles de decisión que votan por la clase más probable.

Características:

- Reduce el sobreajuste.
- Alto rendimiento.
- Difícil de interpretar.

🔸 K-Nearest Neighbors (K-NN)

Clasifica un dato nuevo según las clases de sus "K" vecinos más cercanos.

Características:

- Simple y efectivo.
- Requiere buena elección de K.
- Alto costo computacional para grandes datasets.

🔸 Support Vector Machines (SVM)

Encuentra el hiperplano óptimo que separa las clases.

Características:

- Efectivo para espacios de alta dimensión.
- Requiere ajuste de parámetros.
- Difícil de escalar para grandes volúmenes de datos.

🔸 Redes Neuronales para Clasificación

Permiten resolver problemas de clasificación binaria y multiclase con arquitectura adaptada.

Características:

- Excelente desempeño en problemas complejos.
- Requieren gran cantidad de datos.
- Alta capacidad de ajuste.

Visualización conceptual: Red neuronal para clasificación

```
import networkx as nx
```

```
import matplotlib.pyplot as plt


G = nx.DiGraph()

layers = [3, 4, 3]  # entradas, ocultas, salidas

positions = {}

node = 0


# Crear capas

for i, layer_size in enumerate(layers):

    for j in range(layer_size):

        G.add_node(node)

        positions[node] = (i, -j)

        node += 1


# Conexiones

for i in range(layers[0]):

    for j in range(layers[0], layers[0] + layers[1]):

        G.add_edge(i, j)

for i in range(layers[0], layers[0] + layers[1]):

    for j in range(layers[0] + layers[1], sum(layers)):

        G.add_edge(i, j)


nx.draw(G, pos=positions, with_labels=False, node_size=600,
node_color='lightblue', arrows=False)
```

```
plt.title("Red Neuronal para Clasificación Multiclase")

plt.axis('off')

plt.show()
```

Aplicaciones comunes

- Clasificación de imágenes (CNN)
- Análisis de texto (sentimiento, tema)
- Reconocimiento de voz
- Detección de fraude

12. Dimensiones y la Maldición de la Dimensionalidad

En el contexto del aprendizaje de máquina, el término **dimensión** se refiere al número de variables o características (features) que describen un conjunto de datos. A medida que la cantidad de dimensiones aumenta, también lo hace la complejidad del análisis y del modelo que se necesita para procesar esos datos.

La **maldición de la dimensionalidad** es un fenómeno que ocurre cuando el número de dimensiones en un conjunto de datos es tan alto que afecta negativamente el rendimiento de los algoritmos de aprendizaje de máquina. Este concepto fue introducido por Richard Bellman en 1961 y se presenta, principalmente, cuando el volumen del espacio de búsqueda crece exponencialmente con cada nueva dimensión añadida.

📌 **¿Por qué es un problema?**

Cuando los datos tienen muchas dimensiones:

- La **distancia entre los puntos de datos** pierde significado, ya que todos los puntos parecen estar igualmente lejos entre sí.

- Aumenta el riesgo de **sobreajuste**, dado que los modelos pueden aprender ruido en lugar de patrones reales.

- Se requiere un **mayor volumen de datos** para obtener resultados estadísticamente significativos.

- La **capacidad de generalización** del modelo disminuye.

🔍 **Impacto en los algoritmos**

La maldición de la dimensionalidad afecta, principalmente:

- Modelos basados en **distancias**: como K-NN, donde la noción de cercanía pierde sentido.
- Modelos de **clustering**: los clusters pueden volverse difusos.
- Algoritmos de **regresión y clasificación**: requieren más datos para evitar sobreajuste.

🚀 **Estrategias para mitigar la maldición**

1. **Selección de características (Feature Selection)**
    - Consiste en identificar y conservar las variables más relevantes para el problema.
    - Métodos comunes: pruebas estadísticas, correlación, importancia de características en Random Forest.
2. **Reducción de dimensionalidad**
    - Transformar las características originales en un conjunto más pequeño, manteniendo la mayor cantidad de información posible.
    - Herramientas más utilizadas:
        - **PCA (Análisis de Componentes Principales)**
        - **t-SNE**
        - **LDA (Análisis Discriminante Lineal)**
3. **Normalización y estandarización de datos**
    - Aunque no reduce dimensiones, ayuda a mitigar el efecto del aumento de variables.
4. **Eliminación de variables redundantes**

- Variables que aportan información similar o están altamente correlacionadas.

Cierre ✍️

En este manual, exploramos los fundamentos esenciales del aprendizaje de máquina, sus aplicaciones y los principales conceptos que lo sustentan. Comenzamos por entender qué es el aprendizaje automático y cómo está revolucionando múltiples industrias, desde la medicina hasta las finanzas, gracias a su capacidad para descubrir patrones ocultos en los datos.

Avanzamos luego por las etapas típicas que todo proyecto de aprendizaje de máquina atraviesa, destacando la importancia de la correcta preparación de los datos, la selección del algoritmo y la evaluación de los resultados. También diferenciamos los distintos tipos de aprendizaje automático: supervisado, no supervisado y por refuerzo, permitiendo visualizar sus escenarios de aplicación.

Dedicamos un apartado especial a los algoritmos de regresión y clasificación, pilares fundamentales de la disciplina, detallando ejemplos prácticos y casos de uso que ilustran cómo se utilizan para resolver problemas del mundo real. Finalmente, abordamos un concepto clave para cualquier proyecto de machine learning: la maldición de la dimensionalidad y las estrategias para mitigar sus efectos.

Con estos conocimientos, el lector cuenta con una base sólida para avanzar en el diseño e implementación de soluciones basadas en aprendizaje automático, comprendiendo los beneficios y desafíos asociados.

Referencias📚

- Google Cloud. (s.f.). *What is machine learning?* https://cloud.google.com/learn/what-is-machine-learning
- IBM. (s.f.). *Machine learning.* https://www.ibm.com/cloud/learn/machine-learning
- Microsoft. (s.f.). *What is Azure Machine Learning?* https://learn.microsoft.com/es-es/azure/machine-learning/overview-what-is-azure-machine-learning?view=azureml-api-2
- Scikit-learn Developers. (s.f.). *User guide — scikit-learn documentation.* https://scikit-learn.org/stable/user_guide.html
- Amazon Web Services (AWS). (s.f.). *Learn about machine learning.* https://aws.amazon.com/es/training/learn-about/machine-learning/

¡Muchas gracias!

Nos vemos en la próxima lección

alkemy